Baranyai's theorem on factorisations of complete uniform hypergraphs, formalised in Lean 4 with Mathlib

Josh Bald
Independent researcher, Ontario, Canada
jpbald93@gmail.com

2 October 2026; version 2, 22 October 2026 (adds the Layer-14 and Layer-15 checks of Section 6.1)

Abstract

For positive integers n, k, Baranyai's theorem (1975) says that if k divides n, the $\binom{n}{k}$ k-element subsets of an n-element set can be split into $\binom{n-1}{k-1}$ classes, each of which is a partition of the n-set into n/k blocks. For $k = 2$ this is the classical 1-factorisation of the complete graph K_n, n even. We describe a Lean 4 formalisation, built on Mathlib, of the theorem for vertex set $\{0, \dots, n-1\}$ (the formal colouring statement also admits $n = 0$, where it holds only vacuously), together with its transport to an arbitrary finite type (and to the elements of a `Finset`) and, for $n > 0$, a partition form that spells out the disjointness, covering and class-size properties of the colour classes: each class has n/k blocks (a division lemma that needs $k > 0$), and the number of classes satisfies $\binom{n-1}{k-1} = \binom{n}{k}/(n/k)$ (an identity that needs $n > 0$, $k > 0$ and $k \mid n$, and fails for $n = 0$, $k = 1$). The proof is Baranyai's vertex-by-vertex induction in the form given by Brouwer and Schrijver; the integral rounding step is obtained from Mathlib's Hall marriage theorem, with Hall's condition derived by a weighted double count, so no integral-flow theorem is needed. A converse shows that $k \mid n$ is necessary as soon as one colour class is a perfect matching. The library has 617 lines in three files, contains no `sorry`, and its 32 checked declarations use only Lean's three standard axioms; with the two verification files added in version 2 (restatements in Mathlib's own vocabulary and a comparison with independently written textbook statements) the gate checks 76 declarations, again with standard axioms only. As far as we could determine, within the search scope described in Section 7, no earlier formal proof of the theorem was public.

1 Introduction

Let n and k be positive integers. The *complete k-uniform hypergraph* $K_n^{(k)}$ has n vertices, and every k-element set of vertices is an edge. A *1-factor* (perfect matching, or *parallel class*) of $K_n^{(k)}$ is a set of pairwise disjoint edges that covers every vertex; it has exactly n/k edges, so it can exist only when $k \mid n$. A *1-factorisation* is a partition of all $\binom{n}{k}$ edges into 1-factors. Counting edges, it has $\binom{n}{k}/(n/k) = \binom{n-1}{k-1}$ classes.

Theorem 1 (Baranyai [2])**.** *If $k \mid n$, then $K_n^{(k)}$ has a 1-factorisation: the k-subsets of an n-set can be partitioned into $\binom{n-1}{k-1}$ parallel classes.*

For $k = 2$ the theorem says that K_n has a 1-factorisation when n is even, which has been known since the nineteenth century and is the basis of round-robin tournament schedules: with n

teams, the $n-1$ classes are the rounds, and in each round every team plays exactly one match [11]. The case $k = 3$ ("tournaments of games for three players") was settled by Peltesohn in 1936 [12]. The general case was open until Baranyai proved it in 1973; the proof appeared in the Keszthely colloquium volume of 1975 [2], and Baranyai developed the method further in [3]. The textbook proof, due to Brouwer and Schrijver [4], is reproduced by van Lint and Wilson [15, Ch. 38]. It is an induction on the vertices in which each step is an integral rounding of a fractional assignment, obtained there from the integrality theorem for network flows. In design-theoretic language, the theorem says that the complete design of all k-subsets of an n-set is resolvable whenever $k \mid n$.

The theorem is a textbook example of fractional relaxation followed by integral rounding [15, Ch. 38]. Mathlib's list of "1000+ theorems" contains it: the file `docs/1000.yaml` at the revision used here (`0df444a3`) has the entry `Q790236` with the single field `title: Baranyai's theorem` and no declaration or author recorded [10], and the corresponding page of the 1000+ theorems project lists no formalisation [13]. It is not in Mathlib's list of 100 theorems (`docs/100.yaml`).

Contributions.

- A Lean 4 proof of Baranyai's theorem for the vertex type `Fin n` (Theorem 2), with the number of colours written as $\binom{n-1}{k-1}$ and, in a variant, as $\binom{n}{k}k/n$; and its specialisation to 1-factorisations of K_n.
- Transport of the theorem to an arbitrary finite type of cardinality n and to the elements of a `Finset` (Section 4.1).
- A partition form (Theorem 8) that states the conclusion in terms of colour classes: distinct classes are disjoint, the classes cover all k-subsets, blocks of one class are pairwise disjoint, every class has n/k blocks, and the number of classes is $\binom{n}{k}/(n/k)$ (for $n > 0$, $k > 0$, $k \mid n$); and the converse that $k \mid n$ is necessary.
- A verification package: an axiom gate, six tampering tests, kernel-checked concrete factorisations, and a statement-level audit (Section 6); and, in version 2, the main theorems restated with Mathlib-only types (finite partitions, perfect matchings of the complete `SimpleGraph`) with explicit witnesses, and a comparison with textbook statements formalised by a writer that saw neither code nor paper (Section 6.1).

Availability. The code is at `https://github.com/jpbald93/baranyai`. Line references in this paper are to commit `051ae8c` (`051ae8c4063be0bb70f1726e9f60d0319dd37d91`). The toolchain is Lean `v4.33.1` [8] with Mathlib [9] revision `0df444a3` (tag `v4.33.1`), pinned in `lake-manifest.json`. The library `Baranyai` consists of `Baranyai/Basic.lean` (431 lines), `Baranyai/Transport.lean` (56 lines) and `Baranyai/Partition.lean` (130 lines), with a three-line root file. The four main theorems of `Basic.lean` are declared at the top level; all other declarations of these three files are in the namespace `Baranyai`. Version 2 of this paper adds two verification files, `Baranyai/L14.lean` (409 lines, top-level declarations) and `Baranyai/L15Classical.lean` (358 lines, namespace `Baranyai.Classical`), imported by two new lines at the end of the root file, in a new commit `1ab21623da9572cb10a8630a086c010b9e554269` (Section 6.1). The three library files and every statement of commit `051ae8c` are unchanged in that commit, so all line references below remain valid. Table 1 lists the original development's informal statements with their formal counterparts; Section 6.1 identifies the additional verification statements. Supporting material (the Layer-14 and Layer-15 reports, the blind writer's statements and report, the transcript audit, and the reviews

of this version with the fixes made in response) is in the folder `Baranyai` of the repository `https://github.com/jpbald93/LEAN-verification`.

Table 1: Informal statements and their formal counterparts. Line numbers refer to the first line of the declaration at commit `051ae8c`.

Informal statement	Lean name	Source
Main theorem (Section 2)		
Baranyai's theorem on $\{0,\dots,n-1\}$ (Thm. 2)	`baranyai`	`Basic:396`
The same with $\binom{n}{k}k/n$ colours, $n>0$	`baranyai'`	`Basic:404`
1-factorisation of K_n, n even	`complete_graph_one_factorisation`	`Basic:419`
K_4 splits into 3 perfect matchings	`baranyai_four_two`	`Basic:428`
Proof (Section 3)		
Stage-j invariant (Def. 3)	`Baranyai.Inv`	`Basic:25`
Demand $d_j(S)=\binom{n-j-1}{k-\lvert S\rvert-1}$ for $\lvert S\rvert<k$, else 0	`Baranyai.dem`	`Basic:33`
$(t+1)\binom{N+1}{t+1}=(N+1)\binom{N}{t}$	`Baranyai.choose_key`	`Basic:36`
$(k-\lvert S\rvert)\binom{n-j}{k-\lvert S\rvert}=(n-j)\,d_j(S)$ (Lemma 5)	`Baranyai.weight_eq`	`Basic:40`
$\binom{n-j}{k-\lvert S\rvert}=d_j(S)+\binom{n-j-1}{k-\lvert S\rvert}$ (Lemma 5)	`Baranyai.pascal_dem`	`Basic:52`
$n\binom{n-1}{k-1}=\binom{n}{k}k$	`Baranyai.choose_mul`	`Basic:64`
The slots of a class have total size j	`Baranyai.row_card_sum`	`Basic:71`
$\sum_p(k-\lvert A_{ip}\rvert)=n-j$	`Baranyai.row_weight_sum`	`Basic:78`
Weighted double count (Lemma 6)	`Baranyai.weighted_count`	`Basic:84`
Induction step, via Hall (Lemma 7)	`Baranyai.step`	`Basic:106`
Stage 0: all slots empty	`Baranyai.init`	`Basic:304`
Stage n yields the factorisation (Lemma 4)	`Baranyai.exists_partition_of_inv`	`Basic:325`
An invariant exists at stage n	`Baranyai.exists_final`	`Basic:374`
Transport (Section 4.1)		
Factorisations transport along $\alpha\simeq\beta$	`Baranyai.factorisation_congr`	`Transport:17`
Baranyai for any finite type with $\lvert\alpha\rvert=n$	`Baranyai.baranyai_of_card`	`Transport:43`
Baranyai for the elements of a `Finset`	`Baranyai.baranyai_finset`	`Transport:50`
Partition form (Section 4.2)		
Distinct colour classes are disjoint	`Baranyai.class_disjoint`	`Partition:23`
The colour classes cover all k-subsets	`Baranyai.classes_cover`	`Partition:31`
Two blocks of one colour are disjoint	`Baranyai.block_disjoint`	`Partition:38`
(blocks of a class) $\cdot\, k=\lvert\alpha\rvert$	`Baranyai.class_card_mul`	`Partition:48`
A class has $\lvert\alpha\rvert/k$ blocks, for $k>0$	`Baranyai.class_card`	`Partition:71`
$\binom{n-1}{k-1}=\binom{n}{k}/(n/k)$, for $n>0$, $k>0$, $k\mid n$	`Baranyai.num_classes`	`Partition:78`
Partition form of Baranyai (Thm. 8)	`Baranyai.baranyai_partition`	`Partition:89`
Converse: one perfect-matching class forces $k\mid\lvert\alpha\rvert$	`Baranyai.dvd_card_of_factorisation`	`Partition:106`

2 The theorem and its formal statement

Vertices are the elements of `Fin n`, that is $\{0,\dots,n-1\}$. A k-subset is an element of the subtype `{S : Finset (Fin n) // S.card = k}`, and a candidate factorisation is a colouring `c` of the k-subsets with colours in `Fin M`. The factorisation property is stated directly: for every colour i and every vertex v there is exactly one k-subset of colour i that contains v. The main theorem is `Basic:396`:

```
theorem baranyai (n k : ℕ) (hk : 0 < k) (hkn : k ∣ n) :
```

```
    ∃ c : {S : Finset (Fin n) // S.card = k} → Fin ((n - 1).choose (k - 1)),
      ∀ i (v : Fin n), ∃! S : {S : Finset (Fin n) // S.card = k}, c S = i ∧ v ∈ S.1
```

Theorem 2 (`baranyai`). *Let $0 < k$ and $k \mid n$. There is a map c from the k-subsets of $\{0, \dots, n-1\}$ to $\{0, \dots, \binom{n-1}{k-1} - 1\}$ such that for every colour i and every vertex v exactly one k-subset S has $c(S) = i$ and $v \in S$.*

Reading the statement. Let $n > 0$. The map c assigns each k-subset exactly one colour, and every colour class $c^{-1}(i)$ is non-empty, since it contains a block through the vertex 0; so the classes form a partition of the set of all k-subsets into $\binom{n-1}{k-1}$ (non-empty) parts. The condition "exactly one block of colour i through each vertex" says precisely that the class $c^{-1}(i)$ is a set of pairwise disjoint k-subsets covering every vertex, i.e. a 1-factor. Hence, for $n > 0$, the conclusion is that $K_n^{(k)}$ has a 1-factorisation with $\binom{n-1}{k-1}$ classes, as in Theorem 1. The extreme cases agree with the code: for $k = 1$ there is $\binom{n-1}{0} = 1$ colour, whose class is the set of all n singletons, and for $k = n$ there is $\binom{n-1}{n-1} = 1$ colour, whose class is the single block $\{0, \dots, n-1\}$. Section 4.2 proves the parts of this reading that are not literally in the statement (disjointness of blocks, the size of each class). Natural-number subtraction is truncated in Lean; for $n, k \geq 1$, $n - 1$ and $k - 1$ have their usual meaning.

The case $n = 0$ must be read separately. There are no vertices and no k-subsets, and `(0 - 1).choose (k - 1)` is 1 for $k = 1$ and 0 for $k \geq 2$. For $k = 1$ the code therefore supplies one colour whose fibre is empty; since the parts of an ordinary partition are non-empty, this family of fibres is not a partition with the advertised number of parts. What `baranyai` proves for $n = 0$ is only the valid but vacuous colouring statement: the (empty) map c exists and the condition "for every colour i and every vertex $v \dots$" holds because there is no vertex. `baranyai'`, which writes the number of colours as `n.choose k * k / n`, and the partition form (Theorem 8) assume $0 < n$.

The hypothesis $0 < k$ is not logically required: for $k = 0$ the hypothesis $k \mid n$ forces $n = 0$, and the statement holds vacuously (there is no vertex). It is used by the proof, and with $k = 0$ the theorem would say nothing. The hypothesis $k \mid n$ is necessary, since a single 1-factor partitions the n vertices into k-sets. This is a theorem of the library, `Baranyai.dvd_card_of_factorisation Partition:106`: if some colour class of a colouring of the k-subsets of a finite type α covers every vertex exactly once, then $k \mid |\alpha|$. The tampering test `falsehyp` (Section 6) deletes `hkn` from the statement and checks that the proof no longer compiles; this shows only that the proof uses `hkn`. That the weakened statement is false is a separate fact: for $n = 3$, $k = 2$ three vertices cannot be covered by disjoint pairs, and `scratch/Referee.lean` checks with `decide` that no colouring of the 2-subsets of `Fin 3` with two colours has the factorisation property.

For $k = 2$ the theorem specialises to `complete_graph_one_factorisation Basic:419`: if n is even there is a colouring of the 2-subsets of `Fin n` with $n - 1$ colours in which every colour class is a perfect matching. `baranyai_four_two` is the instance $n = 4$.

Transport and partition statements. `Transport.lean` replaces `Fin n` by an arbitrary finite type `Transport:43`:

```
theorem baranyai_of_card [Fintype α] (n k : ℕ) (hk : 0 < k) (hkn : k ∣ n)
    (hα : Fintype.card α = n) :
    ∃ c : {S : Finset α // S.card = k} → Fin ((n - 1).choose (k - 1)),
      ∀ i (v : α), ∃! S : {S : Finset α // S.card = k}, c S = i ∧ v ∈ S.1
```

and `Partition.lean` states the conclusion as a partition `Partition:89`:

```
theorem baranyai_partition [Fintype α] [DecidableEq α] (n k : ℕ) (hk : 0 < k) (hn : 0 <
    n)
    (hkn : k ∣ n) (hα : Fintype.card α = n) :
    ∃ c : {S : Finset α // S.card = k} → Fin ((n - 1).choose (k - 1)),
      (∀ i (v : α), ∃! S : {S : Finset α // S.card = k}, c S = i ∧ v ∈ S.1) ∧
      (∀ S T, S ≠ T → c S = c T → Disjoint S.1 T.1) ∧
      (∀ i j, i ≠ j →
        Disjoint (univ.filter fun S => c S = i) (univ.filter fun S => c S = j)) ∧
      univ.biUnion (fun i => univ.filter fun S => c S = i) = univ ∧
      (∀ i, (univ.filter fun S => c S = i).card = n / k) ∧
      (n - 1).choose (k - 1) = n.choose k / (n / k)
```

3 Proof outline

We follow the proof of Brouwer and Schrijver [4] as given in [15, Ch. 38], with one change in the rounding step. Write $n = mk$ and $M = \binom{n-1}{k-1}$, and let $[j] = \{0, \dots, j-1\}$. The formal `Baranyai.Inv`, `Baranyai.step` and `Baranyai.exists_partition_of_inv` are stated for arbitrary M and m under their own hypotheses (for `Baranyai.step` only the stage-j invariant `hI : Inv n k m M j A`, $j < n$ and $n = mk$; for `Baranyai.exists_partition_of_inv` only the stage-n invariant), and `Baranyai.block_disjoint` needs no finite-type assumption on the vertices, so the argument below and in Section 4.2 specialises these more general results. Binomial coefficients with a negative lower index do not occur: all lower indices below are natural numbers, and $\binom{a}{b} = 0$ for $b > a$.

Definition 3 (`Baranyai.Inv`, `Basic:25`)**.** A family $A = (A_{ip})$, $i < M$, $p < m$, of subsets of $\{0, \dots, n-1\}$ satisfies the *stage-j invariant* if

1. for each class i the slots $A_{i0}, \dots, A_{i,m-1}$ are pairwise disjoint (`disj`);
2. for each class i their union is $[j]$ (`cover`);
3. every slot has at most k elements (`size`);
4. every $S \subseteq [j]$ with $|S| \le k$ occurs as a slot exactly $\binom{n-j}{k-|S|}$ times, i.e. $\#\{(i,p) : A_{ip} = S\} = \binom{n-j}{k-|S|}$ (`cnt`).

Slots may be empty, and an empty slot is counted once per index (i,p); this is how the formal version realises the multisets of [15]. At stage 0 all slots are empty, and the count condition for $S = \varnothing$ reads $Mm = \binom{n}{k}$, which follows from $n\binom{n-1}{k-1} = \binom{n}{k}k$ (`Baranyai.choose_mul`); this is `Baranyai.init`. At stage n the theorem can be read off:

Lemma 4 (`Baranyai.exists_partition_of_inv`)**.** *If A satisfies the stage-n invariant, then a factorisation with M colours exists: colour each k-subset S by the class i of its slot. (The formal statement asserts only existence.)*

Sketch. At $j = n$ condition 4 says that S occurs $\binom{0}{k-|S|}$ times: once if $|S| = k$ and never if $|S| < k$. Every slot occurs at least once, so every slot has exactly k elements, and every k-subset is exactly one slot (i,p). Colour S by i. Given a colour i and a vertex v, condition 2 gives a slot of class i containing v, and condition 1 makes it unique. □

The induction step. Let $j < n$. Each class must choose one of its slots to receive the new vertex j. Define the *demand* $d_j(S) = \binom{n-j-1}{k-|S|-1}$ if $|S| < k$ and $d_j(S) = 0$ otherwise (`Baranyai.dem`). The step works if each set S is chosen exactly $d_j(S)$ times: then, for $j \in S$, the new count of S is the number of times $S \setminus \{j\}$ was chosen, $d_j(S \setminus \{j\}) = \binom{n-j-1}{k-|S|}$, and for $j \notin S$ it is the old count minus the number of times S was chosen, which is again $\binom{n-j-1}{k-|S|}$ by the next lemma.

Lemma 5 (`Baranyai.weight_eq`, `Baranyai.pascal_dem`)**.** *For $j < n$ and every set S with $|S| \le k$,*

$$(k-|S|)\binom{n-j}{k-|S|} = (n-j)\,d_j(S), \qquad \binom{n-j}{k-|S|} = d_j(S) + \binom{n-j-1}{k-|S|}.$$

The first identity is the absorption identity $(t+1)\binom{N+1}{t+1} = (N+1)\binom{N}{t}$ (`Baranyai.choose_key`, from Mathlib's `Nat.add_one_mul_choose_eq`); the second is Pascal's rule (`Nat.choose_succ_succ'`). In both cases $|S| = k$ is handled separately. The formal `Baranyai.weight_eq` has no size hypothesis at all, and the hypothesis $|S| \le k$ of `Baranyai.pascal_dem` turns out not to be used by its proof (Section 5).

Weighted double counting. In [15] the choice is made by exhibiting a fractional flow, in which class i sends weight $(k-|S|)/(n-j)$ to each of its slots S, and then appealing to the integrality of maximum flows (their Theorem 7.2, or alternatively their rounding theorem for circulations, Theorem 7.4). The formal proof uses the same weights, cleared of denominators. Since the slots of class i partition $[j]$ and there are m of them, $\sum_p (k-|A_{ip}|) = mk - j = n - j$ (`Baranyai.row_weight_sum`, via `Baranyai.row_card_sum`).

Lemma 6 (`Baranyai.weighted_count`)**.** *Assume the stage-j invariant, $j < n$, and let $\mathcal{Y}$ be a family of subsets of $[j]$ of size at most k. Then*

$$\sum_{(i,p)\colon A_{ip} \in \mathcal{Y}} (k-|A_{ip}|) = (n-j) \sum_{S \in \mathcal{Y}} d_j(S).$$

Proof. Group the left-hand side by the value $S = A_{ip}$ (`Finset.sum_fiberwise_of_maps_to`). The fibre of S has $\binom{n-j}{k-|S|}$ elements by condition 4, each contributing $k-|S|$; apply the first identity of Lemma 5. □

Selection by Hall's theorem. For each class i let $t(i)$ be the finite set of *tokens* (S, r) with S a slot of class i and $r < d_j(S)$; in Lean this is `(univ.image (A i)).sigma (fun S => range (dem n k j S))`. A system of distinct representatives for t is exactly a choice of one slot per class in which each S is chosen at most $d_j(S)$ times. Mathlib's Hall theorem, in the form `Finset.all_card_le_biUnion_card_iff_exists_injective` [6, 5], says that such a system exists iff $|X| \le |\bigcup_{i \in X} t(i)|$ for every set X of classes.

Lemma 7 (`Baranyai.step`)**.** *If A satisfies the stage-j invariant, $j < n$ and $n = mk$, then some A' satisfies the stage-$(j+1)$ invariant.*

Sketch. Hall's condition. Let X be a set of classes and $\mathcal{Y}$ the family of sets occurring as slots of classes in X. Then $\bigcup_{i \in X} t(i)$ is the set of tokens (S, r) with $S \in \mathcal{Y}$, $r < d_j(S)$, of size $\sum_{S \in \mathcal{Y}} d_j(S)$. By `Baranyai.row_weight_sum` and Lemma 6,

$$(n-j)|X| = \sum_{i \in X} \sum_{p} (k-|A_{ip}|) \le \sum_{(i,p)\colon A_{ip} \in \mathcal{Y}} (k-|A_{ip}|) = (n-j) \sum_{S \in \mathcal{Y}} d_j(S),$$

and $n - j > 0$. *Exactness.* Hall gives an injective choice $i \mapsto (S_i, r_i) \in t(i)$; let $p(i)$ be a slot of class i with $A_{i,p(i)} = S_i$. Injectivity in r bounds the number of classes choosing S by $d_j(S)$ (`Finset.card_le_card_of_injOn`). Summed over the family $\mathcal{U}$ of all subsets of $[j]$ of size at most k, the numbers of choices add up to M (`Finset.card_eq_sum_card_fiberwise`). Lemma 6 with $\mathcal{Y} = \mathcal{U}$, whose left side is $M(n - j)$, gives $\sum_{S \in \mathcal{U}} d_j(S) = M$. A sum of termwise inequalities with equal totals is a sum of equalities (`Finset.sum_eq_sum_iff_of_le`), so every S is chosen exactly $d_j(S)$ times. A chosen slot has $d_j(S) > 0$ and hence $|S| < k$. *Update.* Put $A'_{i,p(i)} = A_{i,p(i)} \cup \{j\}$ and leave the other slots unchanged. Conditions 1–3 are immediate; condition 4 is the computation before Lemma 5, split according to whether $j \in S$. □

`Baranyai.exists_final` iterates Lemma 7 from `Baranyai.init` up to $j = n$, and `baranyai` combines it with Lemma 4.

Hall instead of flows. Because every class has capacity one and the demand of each set can be split into $d_j(S)$ unit tokens, the integral rounding needed here is a bipartite matching problem, and Hall's theorem suffices. Hall's theorem is itself a consequence of flow integrality, and replacing a capacitated assignment by a matching through token copies is a standard device; we do not claim either as new. Its use here means that no theory of network flows had to be formalised: the integrality of the solution comes from Hall's theorem, and the fractional flow survives only in the double count of Lemma 6.

What was reused and what was built. From Mathlib we use, among others, Hall's marriage theorem (`Finset.all_card_le_biUnion_card_iff_exists_injective`, formalised by Gusakov, Mehta and Miller [5]), the binomial identities `Nat.add_one_mul_choose_eq` and `Nat.choose_succ_succ'`, the `Finset` big-operator lemmas named above together with `Finset.card_biUnion` and `Finset.sum_tsub_distrib`, and, for the transport, the equivalences `Equiv.finsetCongr` and `Fintype.equivFinOfCardEq` and the lemma `Fintype.card_coe`; the proofs also use standard `Finset` and `Nat` lemmas such as `card_image_of_injective`, `card_union_of_disjoint`, `sum_le_sum_of_subset`, `sum_product`, `Fintype.sum_prod_type`, `Fin.card_filter_val_lt`, `card_erase_add_one` and `Nat.choose_eq_zero_of_lt`. Everything else was written for this development: the invariant and the demand function, the arithmetic and double-counting lemmas, the induction step and its initialisation, the extraction of the factorisation, the transport and the partition corollaries.

4 Transport and partition form

4.1 Transport

`Baranyai.factorisation_congr Transport:17` says that if the k-subsets of α have a factorisation with M colours, so do the k-subsets of β, for any bijection $e : \alpha \simeq \beta$. The colour of $T \subseteq \beta$ is the colour of its preimage, computed with `Equiv.finsetCongr`, which preserves cardinality and transports membership. `Baranyai.baranyai_of_card Transport:43` applies this to `Fintype.equivFinOfCardEq` and gives Theorem 2 for any finite type of cardinality n. `Baranyai.baranyai_finset Transport:50` is the case of the subtype $\uparrow s$ of a `Finset` s, with $|s|$ in place of n; the k-subsets are then `Finset`s of elements of s.

These statements add no mathematics: Theorem 1 is about an abstract n-set, and the transport removes the artefact of the vertex type `Fin n`. They make the theorem usable for vertex sets that are

not literally $\{0, \ldots, n-1\}$; the satisfiability file instantiates them for `{1, 5, 7, 9} : Finset` $\mathbb{N}$ and for the type `Fin 2 × Fin 3`.

4.2 Partition form

The lemmas of `Partition.lean` hold for an arbitrary finite type α (for `Baranyai.block_disjoint`, an arbitrary type) and any colouring c of its k-subsets with colours in `Fin M`.

- `Baranyai.class_disjoint`: distinct colour classes $c^{-1}(i)$, $c^{-1}(j)$ are disjoint.
- `Baranyai.classes_cover`: the union of all colour classes is the set of all k-subsets.

These two hold for any function c: the fibres are pairwise disjoint and cover the set of k-subsets, and the non-empty fibres form a partition of it (a colour that is not used has an empty fibre, which can happen for an arbitrary c even when $n > 0$). The next three use the factorisation property `hc`.

- `Baranyai.block_disjoint`: two different k-subsets of the same colour are disjoint.
- `Baranyai.class_card_mul`: every class has b blocks with $b \cdot k = |\alpha|$, by double counting incidences between the blocks of the class and the vertices, each vertex lying in exactly one block. `Baranyai.class_card` divides by k and therefore assumes $k > 0$.

Finally `Baranyai.num_classes` proves $\binom{n-1}{k-1} = \binom{n}{k}/(n/k)$ under the hypotheses $n > 0$, $k > 0$ and $k \mid n$, the edge count of the introduction. The hypothesis $n > 0$ is needed: for $n = 0$, $k = 1$ the left side is $\binom{0}{0} = 1$ and the right side is $\binom{0}{1}/(0/1) = 0$. `Baranyai.baranyai_partition` (Theorem 8) collects all of this into one statement. The same double count gives the converse `Baranyai.dvd_card_of_factorisation`: if one colour class covers every vertex exactly once, then k divides $|\alpha|$.

Theorem 8 (`Baranyai.baranyai_partition`)**.** *Let α be a finite type with n elements, $0 < k$, $0 < n$ and $k \mid n$. There is a colouring c of the k-subsets of α with $\binom{n-1}{k-1}$ colours such that every vertex lies in exactly one block of each colour; two different blocks of the same colour are disjoint; distinct colour classes are disjoint and together contain all k-subsets; every colour class has n/k blocks; and $\binom{n-1}{k-1} = \binom{n}{k}/(n/k)$.*

The first conjunct is the conclusion of Theorem 2; the others follow from it, so Theorem 8 is not a stronger theorem. Its purpose is to make the informal reading of Section 2 checkable: a reader who prefers to see "each class is a set of n/k pairwise disjoint blocks, and the classes partition the edge set" finds exactly that in the statement. As the header of `Partition.lean` says, the hard direction is not reproved.

5 Formalisation notes

Representation. Vertices are `Fin n` in the core development. Edges are `Finset`s with a cardinality constraint, packaged as a subtype, rather than `Set`s; this makes the families of edges finite types and lets `decide` check small instances. A factorisation is a function from edges to colours, so the property "each edge lies in exactly one class" holds by construction and only the 1-factor property has to be stated. In the proof, classes and slots are indexed by `Fin M` and `Fin m`, and the invariant is a structure with four `Prop` fields (`Baranyai.Inv`). Counting slots by index rather than using multisets removes the need to count the empty set with multiplicity. Hall's theorem is applied to the token sets $t(i)$ of type `Finset ((_ : Finset (Fin n)) × ℕ)`, a dependent pair of a set and a token number, so that the injective function produced by Hall encodes both the chosen slot and the multiplicity bound.

Bridges to Mathlib's graph and hypergraph notions. The factorisation property is a direct ∃! statement about `Finset`s, and the three library files do not connect it to Mathlib's `SimpleGraph.Subgraph.IsPerfectMatching`, to the structure `Hypergraph` (which exists at the pinned revision, with `Set`-valued edges), or to `Finpartition`. In particular `complete_graph_one_factorisation` is stated in the same ∃! form. The verification files of version 2 supply the bridges to `Finpartition` and to perfect matchings: `L14.lean` restates the main theorems as families of `Finpartition`s and the case $k = 2$ as a decomposition of the complete `SimpleGraph` into subgraphs satisfying `SimpleGraph.Subgraph.IsPerfectMatching`, and `L15Classical.lean` proves the textbook forms stated with these notions (Section 6.1). No bridge to `Hypergraph` has been formalised. The proof devices `Baranyai.Inv` and `Baranyai.dem` are local and are not bridged to anything; they do not occur in the statements of the main theorems.

An unused hypothesis. The statement-level audit (Section 6) found that the hypothesis `hS : S.card ≤ k` of `Baranyai.pascal_dem` is not used by its proof: the lemma holds without it (when $|S| \geq k$ both sides reduce to $\binom{n-j}{0} = 1 = 0 + 1$ under truncated subtraction).

Size. `Basic.lean` has 431 lines, of which the induction step takes about 200. The two corollary files have 186 lines together. In these three files the definitions are `Baranyai.Inv` and `Baranyai.dem`; all other declarations are theorems. (The Layer-15 file of version 2 adds the writer's four definitions, two bridge definitions, and a proposition-valued definition recording the unproved general almost-regular statement.)

6 Verification

Except for the version-2 checks explicitly identified below, the checks in this section were run on commit `051ae8c`. The library sources, `gate.sh`, `tests/tamper.sh`, `scratch/Satisfiable.lean` and `scratch/Referee.lean` on the build machine were confirmed byte-identical (SHA-256) to those of the commit before the runs.

Axiom gate. `gate.sh` rejects the tokens `sorry`, `admit`, `native_decide`, `axiom`, `#eval`, `run_cmd`, `set_option`, `macro`, `elab`, `syntax`, `notation` and `import Lean` in the library sources, builds the library (8 710 jobs, no errors), generates its own `#print axioms` query for each of 32 fully qualified declarations (every theorem, definition and structure field of the library), and fails on any axiom other than `propext`, `Classical.choice` and `Quot.sound`. It prints `PASS (32 declarations, standard axioms only)`. With the two files of version 2 it prints `PASS (76 declarations, standard axioms only)`, and the tampering tests below are again all rejected (Section 6.1).

Tampering tests. `tests/tamper.sh` plants six defects in scratch copies of the repository and requires the gate to fail on each with a nonzero exit status and the expected message: a `sorry`; an `axiom` split over two lines; an `#eval` printing a fake axiom report; a macro rewriting `#print axioms`; the statement of `baranyai` with the hypothesis `k ∣ n` deleted, which must fail to build (this shows that the proof uses the hypothesis; that the weakened statement is false is checked separately in `scratch/Referee.lean`); and a genuine extra axiom with a long name, so that Lean wraps the report over several lines. All six are rejected (6/6).

Non-triviality and instances. The hypotheses $0 < k$, $k \mid n$ of Theorem 2 are visibly satisfiable, so the real risk is a conclusion that is too weak to fail. Two kernel-checked facts address it. First, `scratch/Referee.lean` (outside the library, exit status 0) proves by `decide` that the conclusion fails at $n = 3$, $k = 2$: no map from the 2-subsets of `Fin 3` to `Fin 2` has the factorisation property. Two colours are the count prescribed by Theorem 2, $\binom{2}{1} = 2$. Second, the library corollary `Baranyai.dvd_card_of_factorisation` shows that the conclusion with at least one colour forces $k \mid n$; `scratch/Referee.lean` applies it to rule out every positive colour count at $n = 3$, $k = 2$. The same file gives a term of the hypothesis of `Baranyai.step`, which carries $j < n$: the stage-0 invariant `Baranyai.init` at $n = 4$, $k = 2$, $m = 2$, with $0 < 4$, fed to `Baranyai.step`. It also checks that three hypotheses used by the proof are logically removable (see the audit below). The file `scratch/Satisfiable.lean`, also outside the library, compiles with exit status 0. It shows that the conclusion is met in concrete cases, which is a weaker point than non-triviality. It contains explicit factorisations for $(n, k) = (4, 2)$ (the matchings $\{01, 23\}$, $\{02, 13\}$, $\{03, 12\}$) and $(6, 3)$ (ten classes, each a triple through 0 together with its complement), each checked by `decide` and not derived from the theorem; the hypotheses $0 < 3$ and $3 \mid 6$ of `baranyai`, checked by `decide`; an explicit stage-4 witness of `Baranyai.Inv` for $n = 4$, $k = 2$, $m = 2$, $M = 3$, meeting the hypothesis of `Baranyai.exists_partition_of_inv`; and instances of `Baranyai.baranyai_finset`, `Baranyai.baranyai_of_card` and `Baranyai.baranyai_partition`. The Python script `code/check_baranyai.py` runs the same induction numerically, with a capacitated bipartite matching in the rounding step, and verifies the resulting factorisations for $(n, k) \in \{(9, 3), (8, 2), (10, 5), (12, 4), (12, 3)\}$.

Statement-level audit. A separate audit tool was run on `Baranyai/Basic.lean` (the same file, identified by its SHA-256). The audit has since been re-run over the complete shipped tree — all three library files, 26 theorems — with the same tool: 93 conclusion mutations, all rejected; no contradictory hypothesis set (24 probed, 2 declarations without hypotheses); every recognised hypothesis classified as needed by its proof except the unused one identified below and the two hypotheses of `baranyai'`, whose control copy does not compile in either run; and no unused-variable warning under the linter, in the three library files or in the three files under `scratch/`. The figures in the three parts below are those of the first run on `Basic.lean`, whose individual mutation diagnostics were adjudicated case by case.

- *Conclusion mutations.* For each of the 15 theorems of the file, small changes are made to the conclusion (flip a relation, negate an equality, change a numeral, and similar) while the proof is kept, and Lean must reject the result. All 55 generated edits failed to compile; none survived. The rejections are of two kinds. In 29 of them the mutated statement is itself ill-typed and the first diagnostic identifies an error in the edited statement: for example, twelve of them render as a malformed application such as `n 1`, one of them as `j 1`, and all seven mutants of `Baranyai.exists_final` are ill-typed (four malformed applications and three dependent-domain mismatches). These 29 say nothing about the proofs. Only the remaining 26 mutants are well-typed statements whose proofs Lean rejects, and only these show that a proof depends on the exact conclusion.

- *Contradiction probe.* For each theorem with hypotheses, automation (`omega`, `decide`, `simp_all`, `linarith` and others) tries to derive `False` from the hypotheses alone, alongside a control that must compile. No hypothesis set was found contradictory (13 probed, 2 theorems without hypotheses). This probe is weak: the probed sets are either simple arithmetic facts or contain `hI : Inv ...`, which the automation cannot unfold. The real

consistency evidence is the stage-0 witness from `Baranyai.init`, fed to `Baranyai.step` in `scratch/Referee.lean`, and the stage-n witness `A42_inv` in `scratch/Satisfiable.lean`.

- *Hypothesis ablation.* Selected, syntactically recognised hypotheses are replaced in turn by `True` in a renamed copy of the declaration, which must then fail to compile. The selector does not cover every hypothesis: it omits, among others, divisibility hypotheses such as `hkn : k ∣ n`, invariant hypotheses such as `hI : Inv ...`, and parity hypotheses such as `hn : Even n`. For this file it produced 19 jobs, of which 17 were executed and 2 skipped: 16 hypotheses were found needed *by this proof* and 1 unused. For `baranyai` the hypothesis `0 < k` is needed by this proof; for `Baranyai.pascal_dem` the hypothesis `S.card ≤ k` is not (Section 5). Ablation does not test whether a statement stays true without the hypothesis. `scratch/Referee.lean` kernel-checks that `baranyai` without `0 < k`, `Baranyai.choose_mul` without `0 < n` and `Baranyai.weight_eq` without `j < n` are still true; these three hypotheses are used by the proofs but are logically removable. `Baranyai.pascal_dem` is merely the one case where the proof does not use its hypothesis. The skipped jobs include the ablation of `baranyai'`, which was skipped because its control copy did not compile. That the proof uses the hypothesis `hkn` of `baranyai` is shown instead by the tampering test above; that it is logically necessary is `Baranyai.dvd_card_of_factorisation`.

The audit also attempts satisfiability certificates for the hypotheses of each theorem. That pass was model-assisted: a local language model, treated as untrusted, proposes concrete values and only Lean decides. In the first run on `Basic.lean` it certified 6 theorems and left 7 unresolved, including `baranyai` and `baranyai'`; in the re-run over the three files it produced no further certificate (18 unresolved, 6 of them failed to produce a candidate). The hand-written `scratch/Satisfiable.lean` covers the main theorem.

Limits of these checks. None of the checks above shows that the formal statement says what the prose says (Section 6.1 adds an independent comparison with textbook statements). The 26 rejected well-typed conclusion mutants show that those proofs depend on the exact conclusions; the other 29 rejections only show that the edits did not typecheck. A failed contradiction probe only shows that automation found no contradiction. Neither check is a proof of satisfiability or of faithfulness. The gate trusts the pinned toolchain, Mathlib and lake configuration. The comparison of formal and informal statements was done by hand: the author checked each statement in Table 1 against its informal counterpart. Section 2 and Theorem 8 are meant to make that comparison easy for the reader.

Name check. The author's script `namegate.sh` (not included in the verification repository) checks that every declaration typeset with `\decl` exists in the local source tree at the commit, in the stated namespace; that every Mathlib name typeset with `\mdecl` is in a list of names checked with `#check` against the pinned Mathlib; and that every source link lands on the declaration line of the declaration it accompanies. It passes on commit `051ae8c` for every name outside Section 6.1; the 15 newly introduced declaration names cited in that section are declared only in the new commit. Run against that commit, the script finds the six top-level ones; the nine in the sub-namespace `Baranyai.Classical`, which its namespace parser does not handle, were confirmed by a direct search of `L15Classical.lean`.

6.1 Layers 14 and 15

The two files of this subsection were added in version 2, in commit `1ab21623da9572cb10a8630a086c010b9e554269`; the three original library source files and their statements, together with the tampering script, are unchanged (the hashes of `Basic.lean`, `Transport.lean`, `Partition.lean` and `tests/tamper.sh` are those of `051ae8c`). On the new commit the library builds, the gate prints `PASS (76 declarations, standard axioms only)` (32 original, 17 from `L14.lean`, 27 from `L15Classical.lean`), the tampering tests are all rejected, and a scan of both files for `sorry`, `admit`, `native_decide`, `axiom`, `set_option`, `macro` and `notation` finds no occurrence outside comments (the gate's own token check also passes).

Layer 14: statements in Mathlib's language and non-vacuity. `Baranyai/L14.lean` restates 8 theorems with Mathlib-only types and proves each from the library through bridges: `baranyai`, `baranyai'`, `complete_graph_one_factorisation`, `baranyai_four_two`, `Baranyai.baranyai_of_card`, `Baranyai.baranyai_finset`, `Baranyai.baranyai_partition` and the converse `Baranyai.dvd_card_of_factorisation`. The colouring is replaced by an injective family P of $\binom{n-1}{k-1}$ `Finpartition`s of the vertex set into k-blocks, each with n/k blocks, such that every k-subset is a block of exactly one P_i (`l14_baranyai`); Theorem 8 becomes one `Finpartition` of the set of k-subsets with $\binom{n-1}{k-1}$ parts, each a partition of the vertices into n/k blocks (`l14_baranyai_partition`); the case $k = 2$ becomes $n - 1$ distinct subgraphs of the complete `SimpleGraph` satisfying `SimpleGraph.Subgraph.IsPerfectMatching`, each edge in exactly one (`l14_complete_graph_one_factorisation`); the converse says that a `Finpartition` of the vertices into k-blocks forces $k \mid |\alpha|$ (`l14_dvd_card_of_factorisation`). A mechanical check of the elaborated types found no constant of the library in any of them. Each theorem has a witness theorem that proves the hypotheses for explicit values and proves the conclusion non-degenerate: for $(n, k) = (6, 3)$, ten pairwise distinct partitions with two triples each (`l14w_baranyai`, and the same for `Fin 2 × Fin 3`); for $(6, 2)$, five distinct partitions with three pairs each; for K_6, five distinct spanning perfect matchings; for K_4, three distinct perfect matchings, and a hand-written factorisation of K_4 checked by `decide` (`l14w_baranyai_explicit`); for $\{0, 2, 4, 6\} \subseteq \mathbb{N}$, three distinct partitions into pairs; for the partition form at $(6, 3)$, ten classes, each with two distinct disjoint triples; and for the converse, a partition of `Fin 6` into two triples, and the consequence that `Fin 5` has no partition into pairs. The other declarations of Table 1 (the invariant, the demand, the arithmetic and counting lemmas, the induction step, `Baranyai.factorisation_congr` and the partition lemmas) are proof internals or are subsumed by the restated theorems, and were not restated. No Mathlib-only statement turned out false, and none needed an extra hypothesis; the only additions are `DecidableEq` instances required by `Finpartition`. The restatement confirms the discussion of $n = 0$ in Section 2: the one-`Finpartition` form genuinely needs $0 < n$ (for $n = 0$, $k = 1$ it would have 0 parts but asserts $\binom{0}{0} = 1$), while the family form `l14_baranyai` holds at $n = 0$ without it.

Layer 15: comparison with textbook statements. Another agent collected three statements from the literature, with citations (Baranyai [2], van Lint and Wilson [15, Thm. 38.1], and the Wikipedia article): Baranyai1, the hypergraph form; Baranyai2, the parallel-class form and its "equivalently" version; and Baranyai3, the general almost-regular form with its parts (a) $k \mid n$ and (b) $k = 2$. An AI model, given only these statements (it saw no code and not this paper; its transcript was audited), wrote them in Lean with Mathlib notions and four definitions of its own (`IsPerfectMatchingK`, `IsPartitionOfKSubsets`, `pointDegree`, `IsAlmostRegular`): 8 statements, all with `sorry`, with three ambiguities marked and no extra hypothesis. A comparer then proved each statement, copied verbatim, from the development, or classified it. `Baranyai/L15Classical.`

`lean` (namespace `Baranyai.Classical`) contains the result. Of the 8 statements, 6 are covered (equivalent to or implied by the library), 1 is a new proof of a statement outside the paper, and 1 is out of scope; 0 are narrower in the library than in the text, and 0 writer statements are false as written.

- *Covered (6):* `Baranyai.Classical.cov_Baranyai1_a` (hypergraph form; the writer's $0 < n$ is not needed), `Baranyai.Classical.cov_Baranyai1_b` ($\binom{n-1}{k-1}n = \binom{n}{k}k$, from `Baranyai.choose_mul`), `Baranyai.Classical.cov_Baranyai2_a` and `Baranyai.Classical.cov_Baranyai2_b` (parallel classes as `Finpartition`s; both readings of the writer's ambiguity at $n = 0$ are covered), `Baranyai.Classical.cov_Baranyai3_a` (the case $k \mid n$ of the almost-regular form), and `Baranyai.Classical.cov_Baranyai3_b` (K_n, n even, as $n-1$ perfect matchings of the complete `SimpleGraph` on any finite type, including $n = 0$).
- *New proof (1):* `Baranyai.Classical.cov_Baranyai3_a_implication`: for $k \mid n$, an almost-regular partition into classes of size n/k consists of perfect matchings (every point degree is 1). The library has no such statement.
- *Out of scope (1):* the general almost-regular form (van Lint–Wilson Theorem 38.1: arbitrary sizes $a_1 + \cdots + a_t = \binom{n}{k}$, k not necessarily dividing n). This is the generalisation that Section 8 lists as not formalised. It is kept as the unproved proposition `Baranyai.Classical.classical_Baranyai3_main`, used and assumed nowhere; only its instance $k \mid n$, $t = \binom{n-1}{k-1}$, $a_i = n/k$ is proved (`Baranyai.Classical.cov_Baranyai3_main_part`).

The bridges written for the comparison (from colourings to their classes, to `Finpartition`s and to Mathlib perfect matchings, and the point-degree computation) are also new proofs. Each proved `classical_...` theorem is the writer's text with only `sorry` replaced, checked byte-for-byte against the writer's file. In two places the library is stronger than the textbook text ($n = 0$ is covered; no $0 < n$ is needed in Baranyai1), and `baranyai'` and `Baranyai.dvd_card_of_factorisation` have no textbook counterpart among the statements. The statement collection, the Lean statements and the comparison were all done by AI agents; no human has checked the textbook statements against the printed sources (the collector notes that its wording of van Lint–Wilson Theorem 38.1 is from memory).

7 Related work

Mathematics. Baranyai's original proof is in [2]; [3] treats edge-colourings of complete hypergraphs more generally. The proof formalised here is Brouwer and Schrijver's [4], as presented in van Lint and Wilson [15, Ch. 38], where it is noted (p. 537) that all known proofs use a form or consequence of their Theorem 7.2 or 7.4, the integrality theorems for flows and circulations. Our rounding step uses one such consequence, Hall's theorem [6], through the token construction of Section 3. Mendelsohn and Rosa survey 1-factorisations of complete graphs [11], and Peltesohn treated the case $k = 3$ [12].

Formalisations. Our searches were made on 2 October 2026. Within Mathlib (source at revision `0df444a3`) the only occurrence of "Baranyai" is the `docs/1000.yaml` entry quoted in the introduction, with no declaration recorded [10]; the theorem is not in `docs/100.yaml`, and Mathlib's `Hypergraph` file defines no matchings or factorisations. The 1000+ theorems project page for `Q790236` lists no formalisation [13]. On GitHub we ran the following searches. Code search for `baranyai` restricted to the Lean language returned no results. Unrestricted code search for

`baranyai` returned results of which we screened the first 50 file paths only for the extensions `.lean`, `.v`, `.thy`, `.ml` and `.miz`, and none had them. Code search for the phrase "Baranyai's theorem" was screened on its first 20 results (repository and file path): apart from the two entries cited above [10, 13], they were prose, data sets, glossaries and lecture notes, and none was a formal proof. Code search for "theorem baranyai", and for "hypergraph factorization" restricted to Lean, returned nothing, and code search for `baranyai` restricted to the file extensions `.v` (Rocq) and `.thy` (Isabelle) also returned nothing. Searches of pull requests and issues of the Mathlib repository for "Baranyai" returned none, and repository search for `baranyai` returned only unrelated repositories named after people. Code search within the GitHub mirror of the Isabelle Archive of Formal Proofs [1], the HOL Light repository [7], and the Rocq repositories `coq-100-theorems`, `math-comp` and `graph-theory` returned nothing, and the list of famous theorems proven in Rocq [14] does not mention the theorem. For Mizar and for the Lean Zulip chat we relied on web search only, including the query `"Baranyai" site:leanprover.zulipchat.com`, which returned no results; the other web searches returned nothing relevant; we did not search the Mizar Mathematical Library or the Zulip chat directly. These searches are not exhaustive. As far as we could determine, no earlier formal proof of Baranyai's theorem was public.

8 Limitations and further work

- *Bridges.* The library statements use a direct ∃! formulation. The version-2 files restate them with `Finpartition` and `SimpleGraph` perfect matchings (Section 6.1); a bridge to Mathlib's `Hypergraph` is still missing, and the bridges live in verification files, not in the library's own API.
- *Generalisations.* Baranyai's method gives more: almost-regular colourings when $k \nmid n$, and factorisations of complete multipartite and non-uniform hypergraphs. None is formalised. The invariant would have to be generalised. The Layer-15 comparison states the general almost-regular form (van Lint–Wilson Theorem 38.1) as an unproved proposition and proves only its divisible instance (Section 6.1).
- *Audit coverage.* The statement-level re-run covers all three library files: mutation, the contradiction probe, ablation and the linter were applied to `Partition.lean` and `Transport.lean` as well, with 93 of 93 mutations rejected. Two theorems yielded no mutants (the mutator produced no conclusion edit for them), and the two hypotheses of `baranyai'` remain unclassified because their control copy does not compile; the per-mutation diagnostics were adjudicated individually only for the first run on `Basic.lean`.
- *Cleanup.* The unused hypothesis of `Baranyai.pascal_dem` could be removed; we left the frozen commit unchanged so that the line references of this paper remain valid.

Acknowledgements and use of AI

A large-language-model AI assistant was used to write the Lean proofs and the Python check, to run the prior-art searches and the audits, and to draft this paper. In version 2, AI agents also wrote the Layer-14 restatements, collected the textbook statements, wrote their Lean versions without access to the code or the paper, and carried out the comparison (Section 6.1). The author directed the work and reviewed the paper, and checked each formal statement in Table 1 against its informal

counterpart by hand. Every proof is checked by the Lean kernel, and the author takes responsibility for all content.

References

[1] *Archive of Formal Proofs*, `https://www.isa-afp.org/`; GitHub mirror `https://github.com/isabelle-prover/mirror-afp-devel`, accessed 2 October 2026.

[2] Zs. Baranyai, On the factorization of the complete uniform hypergraph, in A. Hajnal, R. Rado and V. T. Sós (eds.), *Infinite and Finite Sets* (Colloq. Keszthely, 1973), Vol. I, Colloquia Mathematica Societatis János Bolyai 10, North-Holland, Amsterdam, 1975, 91–108.

[3] Zs. Baranyai, The edge-coloring of complete hypergraphs I, *Journal of Combinatorial Theory, Series B* 26 (1979), 276–294.

[4] A. E. Brouwer and A. Schrijver, Uniform hypergraphs, in A. Schrijver (ed.), *Packing and Covering in Combinatorics*, Mathematical Centre Tracts 106, Mathematisch Centrum, Amsterdam, 1979, 39–73.

[5] A. Gusakov, B. Mehta and K. A. Miller, Formalizing Hall's marriage theorem in Lean, arXiv:2101.00127, 2021.

[6] P. Hall, On representatives of subsets, *Journal of the London Mathematical Society* 10 (1935), 26–30.

[7] J. Harrison, HOL Light repository, `https://github.com/jrh13/hol-light`, accessed 2 October 2026.

[8] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction – CADE 28*, Lecture Notes in Computer Science 12699, Springer, 2021, 625–635.

[9] The mathlib Community, The Lean mathematical library, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.

[10] The mathlib Community, `docs/1000.yaml`, entry `Q790236`, Mathlib repository, `https://github.com/leanprover-community/mathlib4`, revision `0df444a3`, accessed 2 October 2026.

[11] E. Mendelsohn and A. Rosa, One-factorizations of the complete graph—a survey, *Journal of Graph Theory* 9 (1985), 43–65.

[12] R. Peltesohn, *Das Turnierproblem für Spiele zu je dreien*, Dissertation, Berlin; August Pries, Leipzig, 1936.

[13] *1000+ theorems* project, page `_thm/Q790236.md`, `https://github.com/1000-plus/1000-plus.github.io`, accessed 2 October 2026.

[14] *Famous theorems proven in Rocq*, `https://madiot.fr/rocq100/`, accessed 2 October 2026.

[15] J. H. van Lint and R. M. Wilson, *A Course in Combinatorics*, 2nd edition, Cambridge University Press, 2001.